



Programación Orientada a Objetos POO404

Unidad 1



GUIA DE EJERCICIOS #1

1. La siguiente actividad será elaborada **solamente en Grupos de 4 compañeros**.
2. A continuación, se describe un conjunto de ejercicios a ser resueltos en Java, aplicando los principios de la programación orientada a objetos.
3. Cada ejercicio debe ser resuelto en un proyecto diferente y ser resuelto de acuerdo con las condiciones especificadas en la descripción de este.
4. Para evitar descuentos generales en la nota de un ejercicio, se debe cumplir cada una de las restricciones brindadas en la descripción correspondiente.
5. Los identificadores dentro del código deben cumplir las reglas de nomenclaturas usadas por las comunidades de desarrolladores de Java.
6. El código fuente elaborado para la solución se debe documentar con un lenguaje técnico “de programador a programador” (por bloques de líneas y líneas específicas, claves en la lógica de la solución) y de manera impersonal.
7. Cada Ejercicio se solucionará en un Proyecto Java distinto, tomando en cuenta los aspectos siguientes:
 - Antes de la definición de su metodo main, incluir como comentario a la lista de TODOS los integrantes del Equipo que entrega la tarea.
 - Elaborar un diagrama UML con los paquetes, clases (incluyendo la que tenga al método de inicio main) y asociaciones implementadas en su respectiva solución. El UML debe realizarse con el software Draw.IO. El archivo generado (.drawio) debe exportarse también a un archivo de imagen (.png o .jpg); para luego, guardar ambos archivos en la carpeta de solución del proyecto.
 - Comprimir (en .zip o .rar) a la carpeta del proyecto.
 - Elegir a un compañero del Equipo para que suba el archivo comprimido del proyecto de solución por el Enlace de envío creado por su docente en el aula digital.

Importante:

- Cada ejercicio resuelto sera enviado por un integrante distinto del Equipo.
- **Cualquier código “extraño” en la solución presentada (que implemente temas aun no vistos en clase de teoría) o algún elemento que genere sospecha de ser resuelto con el uso de IA, hará que El ejercicio sea ignorado.**

- Si estudiantes no están de acuerdo con la pérdida de nota de un Ejercicio (por sospechas brindadas por docente), deberán defender el Ejercicio y responder, explicar perfectamente a TODAS las preguntas que docente considere necesarias para demostrar la autenticidad del código fuente correspondiente y así recuperar la nota correspondiente.

Ejercicios a Resolver

32% Ejercicio A: Representacion de una Hora

Java posee varias clases para la gestión de fechas dentro de sus App's, entre ellas a la clase `LocalTime`, usada para representar horas, así como las operaciones que se pueden realizar con ellas.

Por lo tanto, se debe implementar una clase que simule parcialmente el funcionamiento de esta clase `LocalTime`, cumpliendo cuidadosamente a los requerimientos descritos a continuación:

- A. Debe contar con los campos necesarios para almacenar una hora (formada por horas, minutos y segundos), tanto en formato de 12 horas como de 24 horas.
- B. Para simplificar el diseño, la clase solicitada limitara la hora almacenada hasta un segundo menos de 24 horas. Cualquier segundo de mas, hará que la hora almacenada se reinicie al valor: 0:00:00 h para así continuar las operaciones.
- C. Encapsular cada atributo y justificar si requiere a solamente getter o un setter o ambos.
- D. Su método constructor específico inicializa sus campos para registrar la hora exacta de las 12 de la madrugada (12:00:00 am o 0:00h).
- E. Sobrecargar el constructor para aceptar una hora específica en 4 parametros, que incluyen la hora, los minutos, los segundos e indicar si es (am) o (pm). Si la hora recibida no es valida, se deja la inicializacion a la hora 12:00:00 am.
- F. Definir otra sobrecarga del constructor para aceptar una hora específica en sus parametros, en formato de 24 horas. Si la hora recibida no es valida, se inicializa a la hora 0:00:00 h.
- G. Crear una función que retorne como String a la hora almacenada en sus campos. El usuario podra elegir con un parametro boolean opcional si la desea ver en formato am/pm o 24 horas. Este parametro valdrá true por defecto (indicando que la hora se presentara en formato am/pm).
- H. Definir un metodo privado que determine si una hora (en formato 12 horas, recibida en 4 parametros) es correcta o no. Retorna true si es valida.
- I. Definir un metodo que permita a usuario actualizar la hora almacenada, enviandola en 4 parametros. Si la hora recibida es valida, se realiza el cambio de la hora interna y retorna (con return) true para confirmar el cambio.

Definir otro metodo similar al descrito en el literal anterior, para actualizar la hora, pero enviada en formato de 24 horas. Si la hora recibida es valida, realiza el cambio de la hora interna y retorna true para confirmar el cambio.

Restricciones:

- Implementar el encapsulamiento apropiado a todos los campos de la clase.
- no esta permitido usar ninguna clase existente de java relacionada al manejo de fechas, de lo contrario, la solución elaborada será descartada.
- Cada método solicitado en el ejercicio y definido en el código, pero que no sea utilizado/invocado, será descartado y se penalizara en la nota de este Ejercicio.

Demostracion:

- Preparar a 3 instancias de la clase modelo para gestión de horas. Para crearlos, utilice un constructor distinto implementado.
- Para confirmar la hora almacenada, por cada objeto, muestre en pantalla a su hora aceptada, tanto en formato am/pm como en 24 h.
- Actualice al objeto 1 con una hora correcta (cualquiera luego del mediodia), al objeto 2 con una hora en formato 24h (cualquiera después de las 8pm) y al ultimo objeto con una fecha 24h incorrecta/invalida.
- Confirmar el cambio de la hora almacenada por cada objeto, mostrando nuevamente en pantalla a su hora aceptada, tanto en formato am/pm como en 24 h.

33% Ejercicio B: Abstraccion de un Conjunto Matematico

Un *Conjunto* es una coleccion de elementos, todos de un mismo tipo y cada uno de los cuales nunca se repiten en su contenido.

Se necesita diseñar en Java a una clase que modele a Conjunto, para almacenar elementos de solamente números enteros ($\mathbb{Z}$).

Para guardar los elementos del conjunto, el objeto debe utilizar un objeto de la clase `ArrayList`.

El objeto debe tener un método publico distinto para resolver a cada una de las siguientes tareas:

- Dos métodos constructores: uno sin parámetros y el otro, sobrecarga del constructor. Este ultimo recibe como parámetro a un vector, conteniendo a los elementos que este tendrá inicialmente.
- Una propiedad de solo lectura que indique la cardinalidad del conjunto almacenado.
- Comprobar si un elemento (recibido en un parámetro) pertenece al contenido actual del conjunto. Si ya existe elemento en el conjunto, retorna true.
- Propiedad de solo lectura, que retorna a un vector con la copia de todos los elementos almacenados en el conjunto.
- Propiedad de solo lectura que retorna un string con la lista de valores almacenados en el conjunto. Esta devolverá “vacío” si conjunto aun no tiene elementos o el listado de valores existentes, con este formato de ejemplo: “ 59 -61 78 -23 0 34”

- F. Insertar un elemento (recibido como parámetro) al conjunto,. Retorna true solamente si pudo insertar valor recibido en el contenido del conjunto.
- G. Remover elemento del conjunto. Recibe en un parámetro al valor a eliminar del conjunto. Retorna true si pudo eliminarlo del contenido del conjunto.
- H. Insertar elementos de un subconjunto. Recibe como parámetro a un objeto (de la misma clase que modela a un conjunto, que esta diseñando en este ejercicio) con los elementos que deben ser insertados al contenido del conjunto. Retorna (con return) a la cantidad de elementos del subconjunto que lograron ser insertados en el contenido.

Restricciones:

Principios de POO:

- Se evaluará cumplimiento de nomenclatura (CamelCase, SnakeCase, etc.) de nombre de la clase y sus elementos, identificación de accesores (get, set).
- Minima redundancia en código. Reutilización interna de miembros de la clase que modela al Conjunto.

Almacenamiento:

- Solo puede utilizar ArrayList para almacenar a los datos del conjunto. Cualquier otra clase de coleccion utilizada para este fin, ocasionara que no se califique todo este ejercicio.

Encapsulamiento:

- Todos los campos de la clase deben tener visibilidad privada, por lo tanto, desde las instancias de esta clase se podra acceder a ellos solamente por los accesores (getter y/o setter) y/o metodos, segun solicite el ejercicio. Cada campo no encapsulado, ocasionara un descuento del 25% en la nota obtenida.
- La clase no puede utilizar a la clase Iterator.

Del objeto ArrayList:

- No utilizar alguna clase de Java orientada a colecciones de datos. Solo se aceptara uso de la clase ArrayList.
- De la clase ArrayList, se permite utilizar solamente a sus métodos add y remove. El uso de cualquier otro miembro (propiedades y otros metodos) de esta clase java.util.ArrayList, ocasionara un descuento del 30% en la nota de este ejercicio.

Calidad de la documentacion interna insertada en el cod. fuente

Diagrama UML de clases:

Cada clase y otros elementos son incluidos. Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta.

Todas las clases y paquetes creados son incluidas en el diagrama. Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta y valor inicial (si lo requiere).

Demostracion:

- En el proyecto se debe definir un único paquete, el cual contendrá la clase que modela el conjunto y a otra clase, donde se definirá a main (con la demostración de funcionamiento).
- Desde main, debe implementarse 3 objetos de la clase solicitada: el conjunto 1 se crea vacío, el conjunto 2 se crea con un vector de 8 valores enteros. El conjunto 3 se crea también con 8 valores, de los cuales, cuatro ya existen en el conjunto 2.
- Debe imprimir en pantalla al listado de valores almacenados en cada objeto, usando el miembro apropiado.
- Invocar el método apropiado del conjunto 2 para insertar en su contenido a los valores del conjunto 3. Luego, imprimir la cantidad de nuevos valores que fueron insertados en conjunto 2. Imprimir nuevamente al listado de valores del conjunto 2.
- Cada método definido en una clase y que no sea invocado, será descartado y se penalizará en la nota de este Ejercicio.

35% Ejercicio C: Registro de recursos audiovisuales

Una mediateca cuenta con un conjunto de libros, revistas, DVDs y CDs de audio y otros recursos, disponibles para sus socios. Se pide realizar un programa en JAVA que permita gestionar el material disponible por la mediateca. El programa debe tener las siguientes opciones:

- Listar materiales disponibles
- Buscar material
- Agregar material
- Modificar material
- Borrar material
- Salir

Los datos asociados a cada tipo de material son los siguientes:

- Para los libros: código de identificación interna, título, autor, número de páginas, editorial, ISBN, año de publicación y unidades disponibles.
- Para las revistas: código de identificación interna, título, editorial, periodicidad, fecha de publicación y unidades disponibles.
- Para los CDs de audio: código de identificación interna, título, artista, género, duración, número de canciones y unidades disponibles.
- Para los DVDs: código de identificación interna, título, director, duración, género.

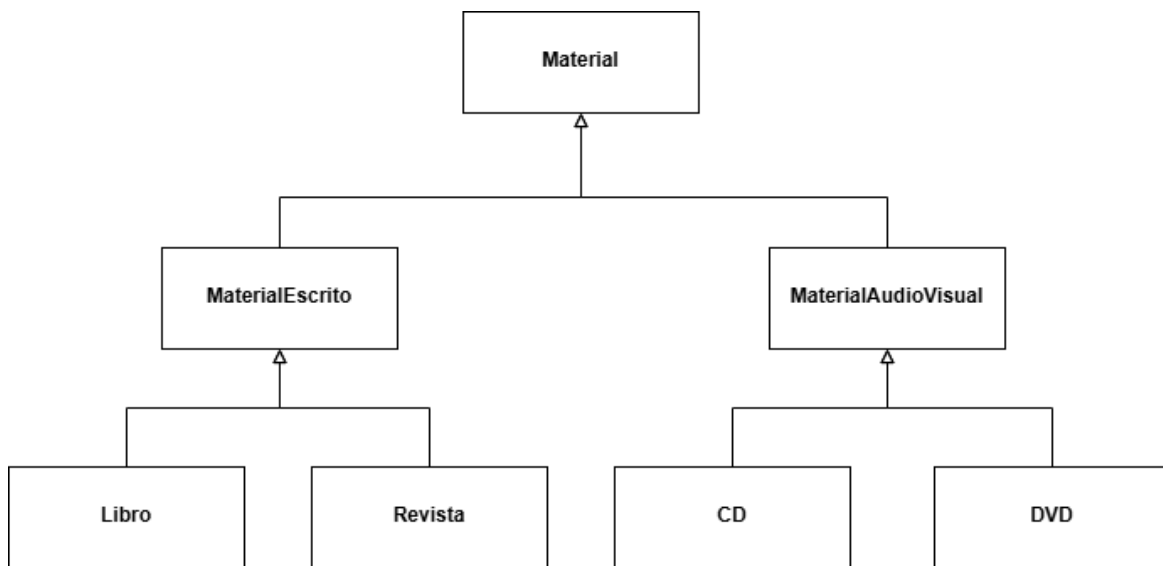
El código de identificación interna es una convención utilizada por la mediateca para identificar de forma única a cada material disponible. Dicho código se genera de la siguiente manera.

- Para los libros: palabra "LIB" más número correlativo de 5 dígitos (Ejemplos: LIB00007, LIB66251, LIB55842).
- Para las revistas: palabra "REV" más número correlativo de 5 dígitos (Ejemplos: REV00257, REV62351, REV59823).
- Para los CDs de audio: palabra "CDA" más número correlativo de 5 dígitos (Ejemplos: CDA00248, CDA12345, CDA55842).

- Para los DVDs: palabra “DVD” más número correlativo de 5 dígitos (Ejemplos: DVD00008, DVD66252, DVD55843).

Consideraciones:

- Para implementar la selección de las diversas opciones de menus, debe hacer la lectura directa del carácter (sin necesidad de presionar Enter para confirmar), utilizando al metodo System.in.read
- Tenga en cuenta las respectivas validaciones para los datos de entrada deben ser realizados por las clases del modelo de gestión de datos, no desde el metodo main.
- Deberá hacer uso de colecciones ArrayList para almacenar los diversos objetos de recursos audiovisuales gestionados por el programa.
- Deberá utilizar programación orientada a objetos, implementando los conceptos de herencia y clases abstractas de tal forma que se implemente la siguiente jerarquía de clases:



Restricciones:

-Implementar el encapsulamiento apropiado (solo lectura, solo escritura o escritura-lectura, según se justifique) a los diferentes campos de la jerarquía de clases.

-Cada método solicitado en el ejercicio y definido en el código, pero que no sea utilizado/invocado, será descartado y se penalizara en la nota de este Ejercicio.

- Debe crearse el Diagrama UML de clases que incluya a todas las clases, desde la que define al metodo estático main hasta el árbol de clases solicitados, actualizados con los miembros implementados en java y las asociaciones correspondientes entre ellas.

- Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta y valor inicial (si lo requiere) creado en el código fuente.

Demostracion:

- Elaborar un menu con las opciones descritas al inicio de este ejercicio.
- Debido a que se crearan objetos de diferentes clases pero todos derivados de una clase base común, se debe crear un único objeto de la clase ArrayList, que almacene el listado de objetos de los diferentes tipos de medios que usuario vaya creando o modificando o los remueva/elimine al usar la app. Investigar uso de palabra instanceof.
- Las verificaciones de datos ingresados por usuario solo deben ser hechas por los setters definidos en el esquema de clases en la descripción de este ejercicio. Es el objeto el que debe indicar que todo esta bien o hay algún tipo de problema y que usuario debe corregir.